

---

# Implementation Document

for

## LoreDrop

Version 1.0

Prepared by

### Group No: 20

Aryan Chaturvedi  
Dhruv Shetty  
Srujan Bhirud  
Vaibhav Itauriya  
Mukund Singhal  
Abhay Tripathi  
Abhay Saxena  
Naveen Godara  
Satwik Nimmagadda

230212  
230370  
231039  
231115  
230670  
230035  
230033  
230685  
230702

### Group Name: Ctrl+Alt+Elite

[aryanc23@iitk.ac.in](mailto:aryanc23@iitk.ac.in)  
[sdhruv23@iitk.ac.in](mailto:sdhruv23@iitk.ac.in)  
[srujansb23@iitk.ac.in](mailto:srujansb23@iitk.ac.in)  
[vaibhav23@iitk.ac.in](mailto:vaibhav23@iitk.ac.in)  
[mukunds23@iitk.ac.in](mailto:mukunds23@iitk.ac.in)  
[abhayt23@iitk.ac.in](mailto:abhayt23@iitk.ac.in)  
[abhaysa23@iitk.ac.in](mailto:abhaysa23@iitk.ac.in)  
[naveeng23@iitk.ac.in](mailto:naveeng23@iitk.ac.in)  
[nsatwik23@iitk.ac.in](mailto:nsatwik23@iitk.ac.in)

**Course:** CS253

**Mentor TA:** *Vinayak Bhosle*

**Date:** 27 March 2026

|                                      |           |
|--------------------------------------|-----------|
| <b>CONTENTS.....</b>                 | <b>II</b> |
| <b>REVISIONS.....</b>                | <b>II</b> |
| <b>1 IMPLEMENTATION DETAILS.....</b> | <b>1</b>  |
| <b>2 CODEBASE.....</b>               | <b>2</b>  |
| <b>3 COMPLETENESS.....</b>           | <b>3</b>  |
| <b>APPENDIX A - GROUP LOG.....</b>   | <b>4</b>  |

## Revisions

| Version                        | Primary Author(s)                                                                                                                                               | Description of Version | Date Completed |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|----------------|
| Initial Draft<br>(Version 1.0) | Aryan Chaturvedi<br>Dhruv Shetty<br>Srujan Bhirud<br>Vaibhav Itauriya<br>Mukund Singhal<br>Abhay Tripathi<br>Abhay Saxena<br>Naveen Godara<br>Satwik Nimmagadda | Initial Draft          | 27/03/26       |

## Implementation Details

LoreDrop is implemented as a full-stack web application using a modern TypeScript-first technology stack, split across a React frontend and a Node.js/Express backend, with MongoDB as the primary database and Firebase for authentication.

### 1.1 Technology Stack Overview

#### 1.1.1 Core Framework: React 19 with TypeScript

React 19 was chosen as the UI library for its component-based architecture, declarative rendering model, and large ecosystem. TypeScript was adopted across the entire frontend to provide static type checking, reduce runtime errors, and improve developer productivity through IDE tooling.

#### 1.1.2 Build Tool: Vite 7

Vite was selected over Create React App (CRA) and Webpack due to its near-instant cold starts using native ES modules and significantly faster Hot Module Replacement (HMR). Vite's Rollup-based production build generates highly optimised bundles with code-splitting.

#### 1.1.3 Routing: React Router DOM v7

React Router v7 provides client-side navigation. It was chosen for its stable API, support for nested routes, and seamless integration with the React ecosystem. It defines routes for /feed, /calendar, /admin, /profile, /auth/callback, and the landing page.

#### 1.1.4 UI Components: Radix UI + Tailwind CSS + shadcn/ui

The component library is built on Radix UI primitives with styling via Tailwind CSS v4 and the shadcn/ui component collection.

**Radix UI:** Provides accessible, unstyled, headless primitives (dialogs, dropdowns, tooltips, tabs, etc.) that guarantee WCAG compliance without bundling opinionated styles

**Tailwind CSS v4:** Utility-first CSS framework preferred over traditional CSS-in-JS because it avoids runtime style injection, produces smaller bundles via PurgeCSS, and offers excellent DX

**shadcn/ui:** Pre-built accessible component recipes on top of Radix + Tailwind, enabling rapid UI development

**Sonner ^2.0.7:** Toast notification system, lightweight alternative to react-toastify

#### 1.1.5 Forms and Validation

**react-hook-form ^7.70:** Performant, uncontrolled form management preferred over Formik because it minimises re-renders by using refs internally

**zod ^3.25:** TypeScript-first schema validation combined with @hookform/resolvers for end-to-end type-safe form validation

#### 1.1.6 Server State Management: TanStack Query v5

TanStack React Query is used for all server state (API data). It provides automatic caching, background refetching, pagination, and a clean separation between server state and local UI state, removing the need for Redux or Context for API data.

#### 1.1.7 Date Handling

**date-fns ^4.1:** Functional date utility library preferred over Moment.js because it is fully tree-shakable and immutable

**react-day-picker ^9.13:** Calendar UI component used in the calendar/saved-events page

### 1.1.8 Authentication: Firebase Auth SDK

Firebase Authentication (firebase ^12.8) is used on the frontend for sign-in/sign-up flows with email + OTP verification. Firebase was chosen over a self-managed JWT system because it handles token refresh, session persistence, and secure credential storage out of the box.

### 1.1.9 Animation

**motion ^12.24 (Framer Motion):** Used for smooth transitions on landing page components and interactive elements

## 1.2 Backend

### 1.2.1 Runtime & Framework: Node.js + Express.js

Node.js with Express.js forms the REST API backend. Express was selected over alternatives such as Fastify or NestJS because of its minimal footprint, simplicity, and large middleware ecosystem.

**express ^4.18.2:** HTTP server and routing

**TypeScript ^5.3.3:** Strong typing for all models and API contracts

**tsx ^4.7:** TypeScript execution for development, replacing ts-node for faster startup

**cors ^2.8.5:** Configures Cross-Origin Resource Sharing between frontend (port 5173) and backend (port 3001)

### 1.2.2 Database: MongoDB with Mongoose

MongoDB was chosen as the primary database for its flexible document model, which maps naturally to the polymorphic data structures in LoreDrop (events with optional recurrence, mixed audience arrays, nested media objects). A relational database would require multiple joins for common queries, whereas MongoDB can return denormalised event documents in a single read.

**mongoose ^7.7:** ODM providing schema definitions, validation, and query building on top of the MongoDB Node.js driver

**Indexes:** Compound and single-field indexes defined on all frequently queried fields (e.g., organizationId, dateTime, isPublished on Event collection)

### 1.2.3 Authentication Middleware: Firebase Admin SDK

The firebase-admin ^12.0 SDK is used on the backend to verify Firebase ID tokens sent in the Authorization: Bearer header. This decentralises credential storage to Google's infrastructure while the backend retains full control over authorisation logic (RBAC).

### 1.2.4 Email Service: Nodemailer

Nodemailer ^7.0 is used for sending OTP verification emails and event-related notifications through an SMTP provider. It was chosen over third-party email APIs (such as SendGrid) to avoid external API billing and allow self-hosting with the institute's SMTP server.

### 1.2.5 Password Hashing: bcryptjs

bcryptjs ^3.0 is used to hash passwords before storage, providing salt-based hashing resistant to rainbow table attacks.

## 1.3 Development Tools & Infrastructure

| Tool                | Version      | Purpose                                                                                              |
|---------------------|--------------|------------------------------------------------------------------------------------------------------|
| pnpm                | latest       | Package manager — faster installs and more efficient node_modules than npm/yarn through hard-linking |
| pnpm workspaces     | workspace v1 | Monorepo management for frontend and backend packages                                                |
| ESLint ^9           | 9.x          | Static code analysis with react-hooks and react-refresh plugins                                      |
| Prettier ^3.7       | 3.x          | Opinionated code formatter for consistent style                                                      |
| TypeScript (strict) | 5.x          | Strict mode enabled across both packages                                                             |
| Vite                | 7.x          | Frontend dev server and bundler                                                                      |

## 1.4 Summary Table

| Layer              | Technology                   | Version     |
|--------------------|------------------------------|-------------|
| Frontend Framework | React                        | 19.2.3      |
| Frontend Language  | TypeScript                   | ~5.9.3      |
| Build Tool         | Vite + SWC                   | 7.3.1       |
| Routing            | React Router DOM             | 7.12.0      |
| UI Primitives      | Radix UI                     | various     |
| Styling            | Tailwind CSS                 | 4.1.18      |
| Component Library  | shadcn/ui (Radix + Tailwind) | -           |
| Forms              | React Hook Form + Zod        | 7.70 / 3.25 |
| Server State       | TanStack React Query         | 5.90        |
| Date Utilities     | date-fns                     | 4.1.0       |
| Auth (Client)      | Firebase Auth SDK            | 12.8.0      |
| Icons              | Lucide React                 | 0.546       |
| Notifications (UI) | Sonner                       | 2.0.7       |
| Animation          | Motion (Framer Motion)       | 12.24       |
| Backend Runtime    | Node.js + Express.js         | 4.18.2      |
| Backend Language   | TypeScript                   | 5.3.3       |
| Database           | MongoDB + Mongoose           | 7.7.0       |
| Auth (Server)      | Firebase Admin SDK           | 12.0.0      |
| Email              | Nodemailer                   | 7.0.13      |
| Password Hashing   | bcryptjs                     | 3.0.3       |
| Package Manager    | pnpm workspaces              | -           |

# Codebase

## 2.1 GitHub Repository

The codebase is hosted on [Github](#).

## 2.2 Repository Structure

| Path                     | Description                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------|
| proj/                    | Monorepo root                                                                                      |
| proj/package.json        | Workspace root with shared scripts                                                                 |
| proj/pnpm-workspace.yaml | pnpm workspace definition listing frontend/ and backend/                                           |
| proj/.env.example        | Template for environment variables (MongoDB URI, Firebase credentials, SMTP config)                |
| proj/eslint.config.js    | Shared ESLint configuration                                                                        |
| proj/md_files/           | Supplementary markdown documentation (ARCHITECTURE.md, FEATURES_COMPLETE.md, EMAIL_SETUP.md, etc.) |
| proj/frontend/           | React + Vite SPA                                                                                   |
| proj/backend/            | Node.js + Express REST API                                                                         |

## 2.3 Frontend Structure

All application source files reside in proj/frontend/src/. The structure follows a feature-based page organisation with shared components extracted to the top-level components/ directory.

| Path                                 | Description                                                                           |
|--------------------------------------|---------------------------------------------------------------------------------------|
| src/main.tsx                         | Application entry point; mounts React to #root                                        |
| src/App.tsx                          | Root component: configures global providers and the React Router route tree           |
| src/lib/api.ts                       | Centralised API service layer; all fetch calls to the backend are defined here        |
| src/lib/utils.ts                     | Shared utility functions (e.g., cn() class name merger)                               |
| src/hooks/use-auth.ts                | Custom hook wrapping Firebase Auth; exposes user, loading, and token accessors        |
| src/components/ui/                   | Shared shadcn/ui component library (Button, Card, Dialog, Input, etc.) ~50 components |
| src/pages/feed/page.tsx              | Main event feed: handles pagination, org filter state, event list                     |
| src/pages/admin/page.tsx             | Organisation admin dashboard: event creation form and event management list           |
| src/pages/calendar/page.tsx          | Saved events calendar view                                                            |
| src/pages/profile/page.tsx           | User profile page with edit form and saved events list                                |
| src/pages/auth/Login.tsx             | Login page; email input trigger for Firebase OTP flow                                 |
| src/pages/auth/EmailVerification.tsx | OTP entry page for new users                                                          |

| Path                   | Description       |
|------------------------|-------------------|
| src/pages/NotFound.tsx | 404 fallback page |

## 2.4 Backend Structure

The backend source lives entirely in proj/backend/src/. Routes are split by resource, with shared concerns (authentication, permissions, email, audit logging) in standalone modules.

| Path                                | Description                                                                                                                       |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| src/server.ts                       | Entry point: initialises Express, CORS, Firebase Admin SDK, connects Mongoose, registers all route groups                         |
| src/models.ts                       | All Mongoose schema and model definitions: User, Organization, Event, EventUpvote, EventComment, CalendarSave, Notification, etc. |
| src/middleware.ts                   | Express middleware: authMiddleware (requires valid Firebase Bearer token), optionalAuthMiddleware                                 |
| src/permissions.ts                  | RBAC helper functions: checks whether a given userId has admin/moderator/member role in a given organisation                      |
| src/email.ts                        | Nodemailer transporter initialisation and helper functions for sending OTP and notification emails                                |
| src/audit.ts                        | Audit log helper: records actor, action, entity type, and metadata to the AuditLog collection                                     |
| src/routes/auth.ts                  | Authentication routes: POST /api/auth/send-verification-code, POST /api/auth/verify-code, POST /api/auth/complete-profile         |
| src/routes/events.ts                | Event CRUD: GET /api/events/feed, GET /api/events/upcoming, GET /api/events/:id, POST /api/events                                 |
| src/routes/interactions.ts          | User interactions: upvote toggle, comment CRUD, calendar save toggle, notification read/unread                                    |
| src/routes/organizations.ts         | Organisation listing: GET /api/organizations, GET /api/organizations/:slug                                                        |
| src/routes/organization-requests.ts | Organisation access requests: POST (user requests), GET (admin views pending), PATCH (admin approves/rejects)                     |

## 2.5 Environment Configuration

Both packages use .env files for environment-specific configuration. An .env.example at the monorepo root documents all required variables. Key variables include:

1. MONGODB\_URI: Connection string for the MongoDB instance
2. FIREBASE\_PROJECT\_ID / FIREBASE\_PRIVATE\_KEY / FIREBASE\_CLIENT\_EMAIL: Firebase Admin SDK service account credentials (backend)
3. VITE\_FIREBASE\_API\_KEY / VITE\_FIREBASE\_AUTH\_DOMAIN: Firebase client SDK configuration (frontend)
4. SMTP\_HOST / SMTP\_PORT / SMTP\_USER / SMTP\_PASS: Nodemailer email transport configuration
5. VITE\_API\_URL: Base URL for the backend API, used by the frontend api.ts service layer

## 2.6 How to Run

From the monorepo root:

*# Install all dependencies*

**pnpm install**

*# Start backend (port 3001)*

**cd backend && pnpm dev**

*# Start frontend (port 5173) in a new terminal*

**cd frontend && pnpm dev**

To seed the database with sample data: `cd backend && pnpm seed`



## Completeness

### 3.1 SRS Functional Requirements Implementation

| SRS Ref.      | Requirement                                                                      | Status   | Implementation Location                                                                                |
|---------------|----------------------------------------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------|
| F1            | Campus-only access via IITK email; role-based user profiles                      | Complete | backend/routes/auth.ts; Firebase Admin SDK verification; models.ts (User schema with role field)       |
| F2            | Organizer verification workflow; admin approve/reject                            | Complete | backend/routes/organization-requests.ts; backend/src/email.ts; admin/page.tsx                          |
| F3            | Organizers create / edit / delete event posts; draft support; audience targeting | Complete | backend/routes/events.ts; admin/page.tsx; models.ts (Event schema)                                     |
| F4 (Feed)     | Scrollable feed with event cards; sort/filter; upcoming events sidebar           | Complete | pages/feed/page.tsx; feed/_components/EventCard.tsx; GET /api/events/feed                              |
| F4 (Calendar) | Calendar view for chronological event browsing                                   | Complete | pages/calendar/page.tsx; react-day-picker; GET /api/interactions/calendar/saved/all                    |
| F4 (Detail)   | Dedicated event detail page with full description, media, comments, upvotes      | Complete | EventCard expanded view with comments section; GET /api/events/:id                                     |
| F5            | Keyword-based event search; tag and organization filters                         | Partial  | OrganizationFilter component (complete); EventFilters.tsx (UI built); full-text keyword search pending |
| F6            | Follow organisations; personalised feed; notifications for followed orgs         | Complete | Organization subscription model in models.ts; follow/unfollow UI; notifications implemented            |
| F7 (Upvote)   | Upvote / toggle; count display                                                   | Complete | POST /api/interactions/upvote/:eventId; EventCard heart button; EventUpvote model with unique index    |

| SRS Ref.     | Requirement                                                                      | Status   | Implementation Location                                                                                                                |
|--------------|----------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------|
| F7 (Comment) | Threaded comments under each event; count display                                | Complete | POST/GET /api/interactions/comments/:eventId; EventComment model                                                                       |
| UC01         | Upload event with validation, venue/time conflict check, audience visibility     | Complete | backend/routes/events.ts; admin/page.tsx; RBAC via permissions.ts                                                                      |
| UC02         | Add event to personal calendar; clash warning; Google Calendar opt-in            | Partial  | POST/GET /api/interactions/calendar/:eventId (complete); Google Calendar API integration pending                                       |
| UC03         | Student registration with IITK email OTP verification                            | Complete | backend/routes/auth.ts; frontend/EmailVerification.tsx; VerificationCode model with TTL index                                          |
| UC04         | Organisation account request and admin approval workflow                         | Complete | backend/routes/organization-requests.ts; admin/page.tsx; OrganizationRequest model; email notification                                 |
| UC05         | Follow an EventAdmin / organisation; toggle follow; personalised feed update     | Partial  | Subscription model defined; full follow-toggle on org profile pending                                                                  |
| UC06         | Filter and search events by keyword, date, category, department, organising body | Partial  | Organisation filter complete; date/category filters UI built; backend support for full multi-parameter filtering partially implemented |
| UC07         | Edit or cancel events; mark cancelled; notify interested users                   | Partial  | Edit form in admin/page.tsx present; PATCH /api/events/:id not yet fully exposed; cancellation status field exists in schema           |

### 3.2 Non-Functional Requirements

| NFR Category | Requirement Summary                   | Status   | Notes                                                                                                 |
|--------------|---------------------------------------|----------|-------------------------------------------------------------------------------------------------------|
| Performance  | Feed page < 2s normal load, < 4s peak | Complete | MongoDB compound indexes on (organizationId, dateTime, isPublished); pagination on all feed endpoints |

| NFR Category       | Requirement Summary                                            | Status   | Notes                                                                                                                      |
|--------------------|----------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------|
| Performance        | 1500 concurrent active users                                   | Partial  | Architecture is stateless and horizontally scalable; load testing not yet performed                                        |
| Security – Auth    | HTTPS; IITK email only; RBAC enforcement                       | Complete | Firebase token verification on every protected route; email domain validation in auth.ts; RBAC helpers in permissions.ts   |
| Security – Data    | Passwords hashed; PII encrypted in transit                     | Complete | bcryptjs hashing; HTTPS enforced; TLS in transit                                                                           |
| Security – Attacks | Brute force / SQL injection / XSS protection                   | Complete | MongoDB (no SQL injection surface); Mongoose schema validation; CORS configured; rate limiting ready but not yet activated |
| Usability          | Core actions within 3 interactions                             | Complete | Feed → event card → upvote/comment achievable in 2 interactions                                                            |
| Availability       | 99% uptime during semester                                     | Partial  | Modular design isolates email service outages; deployment config included                                                  |
| Reliability        | MTTF >= 1 week; data not lost on partial failure               | Complete | MongoDB Atlas with automatic backups planned; persistent data layer                                                        |
| Maintainability    | Modular architecture; separate auth/event/notification modules | Complete | Each route file is a standalone Express Router; models.ts centralises schema                                               |
| Portability        | Deployable on institute server; standard browser support       | Complete | Standard web technologies; no native dependencies; browser list in SRS supported by Radix/Tailwind                         |

### 3.3 Features Implemented — Summary

| Feature                                                    | Status   |
|------------------------------------------------------------|----------|
| Landing page with feature overview and platform statistics | Complete |
| IITK email OTP sign-up and verification                    | Complete |
| Email + password login with Firebase token-based session   | Complete |
| Role-based access control (Student / Organizer / Admin)    | Complete |
| Organisation admin dashboard for event creation            | Complete |
| Event feed with organisation filter and pagination         | Complete |
| Upcoming events sidebar                                    | Complete |
| Event upvote / unlike toggle                               | Complete |
| Event comment section (add + view)                         | Complete |
| Save event to personal calendar (bookmark)                 | Complete |
| Saved events calendar view (/calendar)                     | Complete |
| User profile page with edit functionality                  | Complete |

| Feature                                                    | Status   |
|------------------------------------------------------------|----------|
| In-app notification centre with unread badge (10s polling) | Complete |
| Organisation access request flow                           | Complete |
| Admin dashboard for approving / rejecting org requests     | Complete |
| Light / dark theme toggle with persistence                 | Complete |
| Responsive layout (desktop + mobile hamburger menu)        | Complete |
| Audit logging for sensitive actions                        | Complete |
| Database seed script for development                       | Complete |
| Follow / unfollow organisation toggle UI                   | Complete |
| Full-text keyword event search                             | Partial  |
| Event edit and cancellation from admin dashboard           | Partial  |
| Google Calendar integration                                | Partial  |
| Draft event save before publish                            | Pending  |
| Engagement analytics for organizers                        | Pending  |
| Recurring events                                           | Pending  |

### 3.4 Future Development Plan

The following features are planned for upcoming versions of LoreDrop, in approximate priority order.

#### Version 1.1 — Short-Term Enhancements

6. **Full-text keyword search:** Implement MongoDB text index on Event.title and Event.description; wire up the search bar query parameter in GET /api/events/feed; add front-end debounce and result highlighting
7. **Event edit and cancellation:** Expose PATCH /api/events/:id route; update admin dashboard with edit form pre-filled with existing event data; send cancellation notifications to users who upvoted or saved the event
8. **Follow / unfollow organisation toggle:** Add a dedicated OrganizationFollow model; expose POST/DELETE /api/organizations/:id/follow endpoints; update the 'For You' feed query to filter by followed organisations
9. **Advanced multi-parameter filtering:** Add date range, tag, department, and mode (online/offline) filters to the feed query; update EventFilters.tsx to pass all parameters

#### Version 1.2 — Medium-Term Features

10. **Event draft save:** Add a 'Save as Draft' button on the create-event form; drafts are stored with isPublished: false and visible only to the organiser
- Real-time WebSocket updates:** Replace the current 10-second polling mechanism with Socket.IO for push-based feed and notification updates, reducing unnecessary network traffic
11. **Google Calendar integration:** Complete the UC02 Google Calendar opt-in using the Google Calendar API; store OAuth tokens per user; create calendar events on save
12. **Email notifications:** Send email digests to users for new events from followed organisations using the existing Nodemailer infrastructure

### Version 1.3 — Long-Term Features

13. **Engagement analytics dashboard:** Surface upvote trends, comment counts, view counts, and RSVP numbers to organizers via a dedicated analytics page; implemented with Recharts
14. **Recurring events:** Activate the recurrence fields in the Event schema; generate occurrence instances from a recurrence rule; display recurring events correctly on the calendar
15. **Trending event sort:** Implement a trending score based on upvote velocity and comment activity; expose as a sort option in the feed
16. **Mobile application:** Develop a React Native companion app reusing the existing REST API, targeting iOS and Android
17. **SSO with IITK LDAP:** Replace or augment the current email-OTP login with IITK's institutional Single Sign-On (SSO/LDAP) once institutional access is granted

## Appendix A - Group Log

- A WhatsApp group was created for communication between members.
- Offline meetings were hosted whenever necessary to take inputs from members on the project progress.

| Date and Time duration of the Meet | Meet Type    | Discussions in the meet                                                                                                        |
|------------------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------|
| 11/03/2025<br>9:00 PM - 12:00 PM   | Offline Meet | Meet to discuss the initial implementation idea.                                                                               |
| 13/03/2025<br>9:00 PM - 12:00 PM   | Offline Meet | Tasks associated with the initial draft were equally distributed to each team member.                                          |
| 18/03/2025<br>9:00 PM - 2:00AM     | Offline Meet | A meeting was scheduled to discuss and review the initial draft.                                                               |
| 21/03/2025<br>10:00 AM - 12:00 AM  | Offline Meet | A meeting was held to make changes to the initial draft, ensuring that each team member contributed equally to its completion. |
| 27/03/2025<br>9:00 PM - 1:00 AM    | Offline Meet | A meeting was held to prepare the final draft, ensuring that each team member contributed equally to its completion.           |